Accurate localization and estimation of a sphere in a photo

Dario Comanducci

1 Introduction

A sphere is a special type of quadric, and as such, the optical cone tangent to the quadric defines on it a *contour generator* composed of a conic whose projection onto the image plane (the *occluding contour*) retains the status of a conic [8, pp. 73-74]; in the case of the sphere, the contour generator is a circle and the occluding contour is generally an ellipse [12].

The accurate estimation of such an ellipse is the objective of this work: through a multi-resolution approach (i.e., using a *pyramid* of scaled images, Fig 1), starting from an initialization as a circle on the lowest-resolution image (under normal conditions, a photographed sphere appears at first approximation as a circle), the ellipse estimation is then refined progressively as the image is restored to its original dimensions. The ellipse refinement can be carried out using various methods.[1]

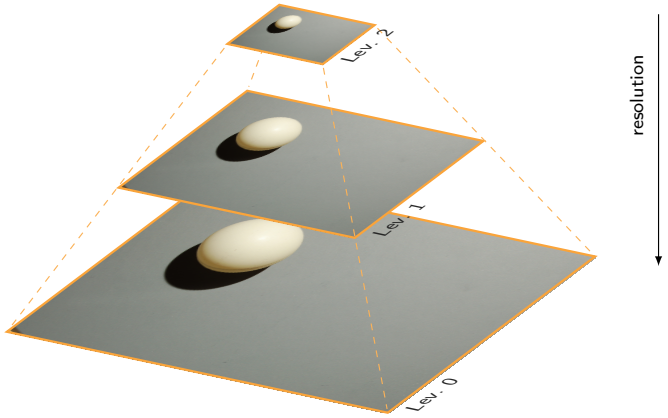


Fig. 1. Multi-resolution pyramid: at the highest level lies the lowest-resolution image; descending down the pyramid, the resolution gradually increases until reaching the original image at level 0.

2 Multi-resolution estimation

2.1 General considerations

The algorithm used to generate images at decreasing resolutions is called the *pyramid of Gaussians*[2] [1]; it is based on convolving the image with an (approximately) Gaussian 5×5 kernel, which acts as an anti-aliasing filter, followed by downsampling with a factor of 2: referring back to Fig 1, level 0 of the pyramid corresponds to the original image; at level 1 the image has half the dimensions of the previous one, and so on down to level $L-1$.

[1] The entire implementation was carried out in Python, employing the OpenCV library in certain steps of the estimation algorithm.

[2] Implemented in OpenCV as `cv.pyrDown`.

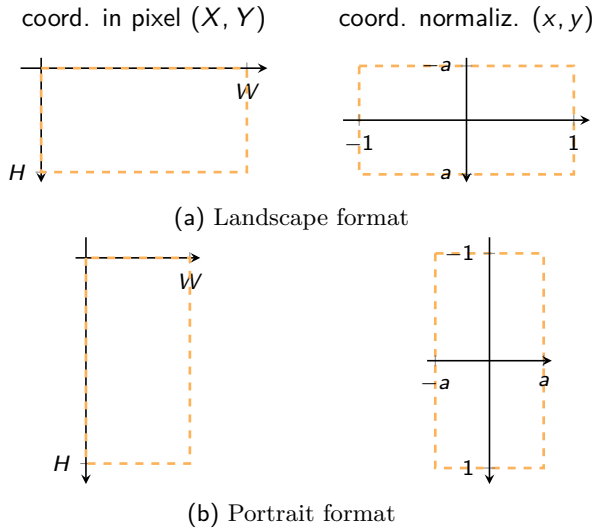


Fig. 2. Mapping from pixel coordinates to normalized coordinates.

2.1.1 Normalized coordinates

To facilitate working with images at different resolutions, a variant of normalized coordinates used in computer graphics is adopted: given an image with dimensions $W \times H$ pixels, the coordinates of each pixel are mapped to $[-1, 1]$ along the longer axis and $[-a, a]$ along the shorter one, where $a = \min(H, W)/\max(H, W)$ is the inverse of the aspect ratio [11], as shown in Fig. 2. Denoting the integer pixel coordinates as (X, Y), the normalized coordinates (x, y) are obtained as:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} 2/s & 0 & -W/s \\ 0 & 2/s & -H/s \\ 0 & 0 & 1 \end{bmatrix}}_{\mathtt{T_N}} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} \tag{1a}$$

$$s = \max(W, H) \tag{1b}$$

The inverse transformation, defined by the matrix $\mathtt{U_N} = \mathtt{T_N}^{-1}$, finally allows for converting the ellipse equation from normalized coordinates back into the integer pixel coordinates (see Eq 5, which exploits the homogeneous representation of a conic).

2.1.2 Mathematical modeling

Ellipses are specific types of conics[3], and therefore must satisfy the equation

$$ax^2 + bxy + cy^2 + dx + ey + f = 0 \tag{2}$$

with a, b, and c not simultaneously zero; in order for Eq. (2) to represent an ellipse, the following constraint must also hold [4][5]:

$$b^2 - 4ac < 0 \tag{3}$$

Matrix representation. Eq. (2) can be condensed into matrix form by representing the conic in *homogeneous* coordinates: let $\mathbf{q} = [x\ y\ 1]^\top$, then Eq (2) is equivalent to [8, p. 30]

$$\mathbf{q}^\top \underbrace{\begin{bmatrix} a & b/2 & d/2 \\ b/2 & c & e/2 \\ d/2 & e/2 & f \end{bmatrix}}_{\mathtt{C}} \mathbf{q} = 0 \tag{4}$$

The matrix $\mathtt{C}$ is defined up to a scale factor, since so are the parameters $a \dots f$ of Eq. (2); in what follows, the ellipse will often be identified by this matrix.

This representation allows for compact recovery of the ellipse parameters in pixel coordinates, once they have been computed from points in normalized coordinates: referring to Eq. (1), let $\mathtt{U_N} = \mathtt{T_N}^{-1}$ be the matrix modeling the inverse transformation (from normalized to pixel coordinates); if $\mathtt{C}$ is the ellipse matrix in normalized coordinates, then the matrix $\mathtt{C}'$ with parameters relative to pixel coordinates is obtained as [8, p. 37]

$$\mathtt{C}' = \mathtt{U_N}^{-\top}\, \mathtt{C}\, \mathtt{U_N}^{-1} = \mathtt{T_N}^{\top}\, \mathtt{C}\, \mathtt{T_N} \tag{5}$$

Sampson error. The computation of the distance of a point $q = (x, y)$ from an ellipse can also be compactly described using the matrix representation $\mathtt{C}$ [8, pp. 97–100]:

$$\delta_\perp^2(q, \mathtt{C}) = \frac{(\mathbf{q}^\top \mathtt{C}\, \mathbf{q})^2}{4\left((\mathtt{C}\mathbf{q})_1^2 + (\mathtt{C}\mathbf{q})_2^2\right)} \tag{6}$$

Fig. 3 shows the values assumed by $\delta_\perp(q, \mathtt{C})$ and by the algebraic error $\varepsilon(q, \mathtt{C})$ for the ellipse defined by the equation $ax^2 + cy^2 = ac$, compared with the true geometric distance $d_\perp(q, \mathtt{C})$; for consistency with $\varepsilon(q, \mathtt{C})$, which takes negative values inside the ellipse, $d_\perp(q, \mathtt{C})$ and $\delta_\perp(q, \mathtt{C})$ are also reported with sign. In the case considered (ellipse with equation $4x^2 + 9y^2 = 36$), the Sampson error is comparable to the true distance for $\delta_\perp(q, \mathtt{C}) < 0.5$ (see also Fig. 4).

Eq. (6) is actually an approximation of the geometric distance[4] between a point and an ellipse, and is referred to as the *Sampson error*; in turn, $\delta_\perp$ can be interpreted as a normalization of the *algebraic error*.

$$\varepsilon(q, \mathtt{C}) = \mathbf{q}^\top \mathtt{C}\, \mathbf{q} = ax^2 + bxy + cy^2 + dx + ey + f \tag{7}$$

by the scaling factor $2\sqrt{(\mathtt{C}\mathbf{q})_1^2 + (\mathtt{C}\mathbf{q})_2^2}$, which depends on the point $q(x, y)$.

[3] Besides ellipses, disregarding degenerate cases, Eq. (2) also describes parabolas and hyperbolas.

[4] An algorithm computing the true geometric distance is described in Appendix A.

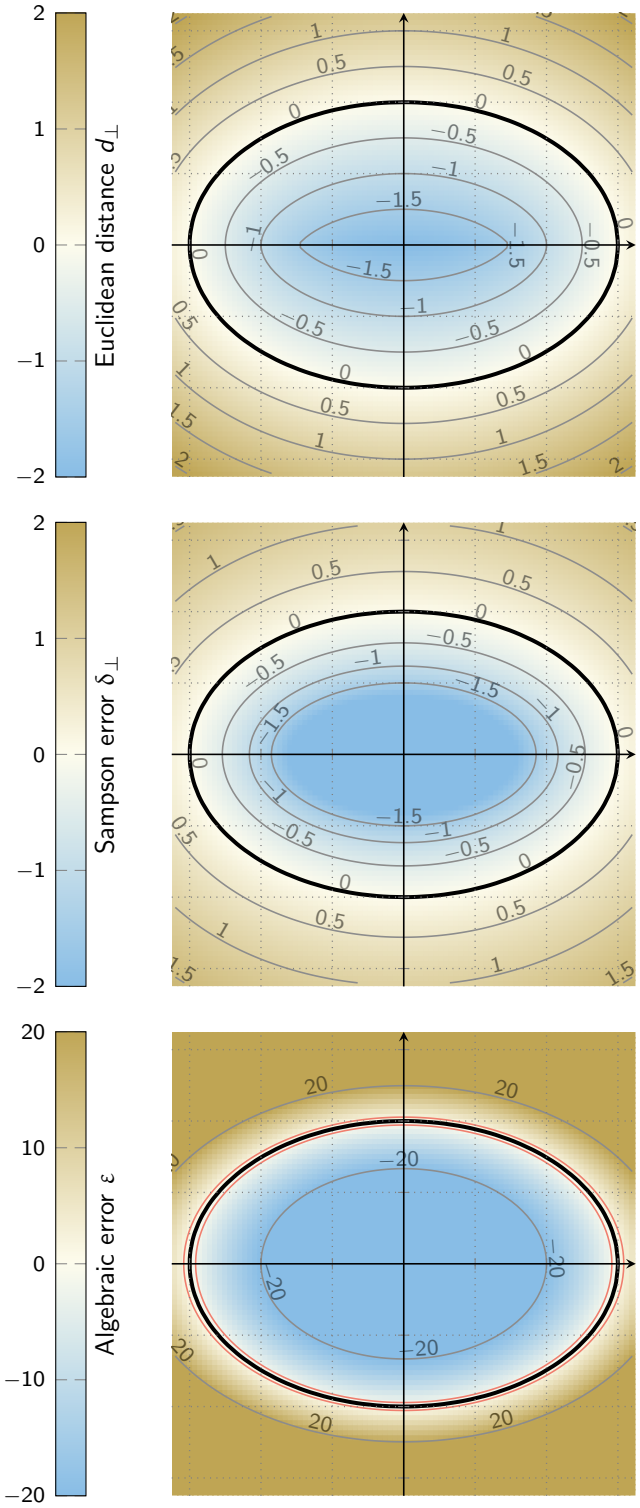


Fig. 3. Geometric distance $d_\perp$ compared with Sampson error $\delta_\perp$ and algebraic error ε with respect to the ellipse $4x^2 + 9y^2 = 36$. The two red level curves for the algebraic error near the ellipse bound the region where $|\varepsilon| \le 2$; note that the scale for ε is ten times larger than that of $d_\perp$ and $\delta_\perp$.

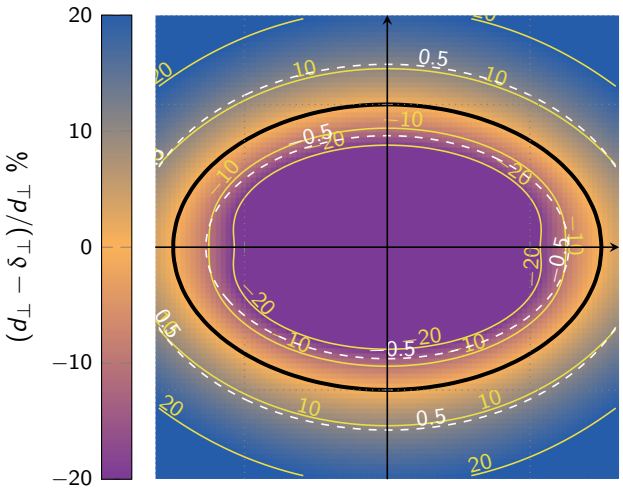


Fig. 4. Percentage variation between $d_\perp$ and $\delta_\perp$; the dashed (white) contour line for $d_\perp = \pm 0.5$ is almost overlapped with the percentage variation of $\pm 10\%$.

The circle as a conic. We conclude with the adaptation of Eq. (2) for the case of a circle with center (x_c, y_c) and radius r:

$$(x-x_c)^2+(y-y_c)^2 = x^2+y^2-2x_cx-2y_cy+x_c^2+y_c^2-r^2 = 0$$

It follows that for a circle, $a = c = 1$, $b = 0$, $d = -2x_c$, $e = -2y_c$, $f = x_c^2 + y_c^2 - r^2$, and in matrix form we have

$$\mathsf{C} = \begin{bmatrix} 1 & 0 & -x_c \\ 0 & 1 & -y_c \\ -x_c & -y_c & x_c^2 + y_c^2 - r^2 \end{bmatrix} \tag{8}$$

2.1.3 Processing at each level

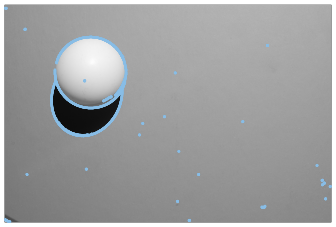
At each level of the pyramid, contours (or *edges*, Fig. 5) are extracted using the Canny method[5] [2]. Going down the pyramid, the ellipse model estimated at one level allows restricting the area where edges are extracted in the next level, leading to substantial gains in computation time by avoiding the application of Canny over the entire image.[6]

The points forming the edges are treated as an unstructured cloud of independent points (i.e., their grouping into continuous segments is not considered), within which the points that most likely describe the sphere in the image are identified (we will refer to these points as *inliers*); an accurate ellipse fitting is then performed on this selection. In contrast to the inliers, the edge points that are too far from the ellipse (we will refer to these points as *outliers*) are discarded during the estimation process.

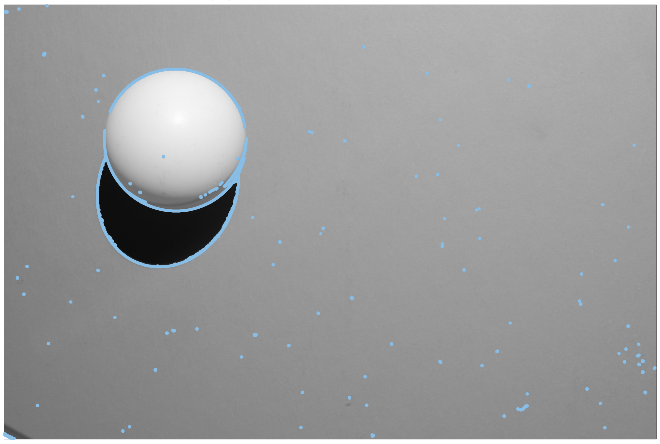
[5] Algorithm available in OpenCV via `cv.Canny(img, thr1, thr2)`: similarly to its use in `cv.HoughCircles` (see § 2.2), by default `thr2=thr1/2`; the value of the first threshold was set to `thr1=100` and kept constant across all levels.

[6] In practice, Canny is applied to the entire image only at the top level of the pyramid; in other levels, Canny is executed only on a rectangular region containing the ellipse estimated at the immediately preceding level.

(a) 752×500 pixel



(b) 1504 × 1000 pixel



(c) 3008 × 2000 pixel

Fig. 5. Edge points (in light blue) for the pyramid in Fig. 1 extracted by Canny over the entire image at each level: 6430 edge points at level 0 (c); 2546 at level 1 (b); 1243 at level 2 (a).

2.2 Initialization

Taking as an example the edges in Fig. 5, note that they contain not only the contour of the sphere but also that of the shadow—also describable as an ellipse—as well as smaller spurious edges[7] scattered here and there due to dust; however, the pyramid downsampling process ensures that in the image at the highest level (level 2, with the lowest-resolution image) such spurious edges have been "automatically" eliminated.

The reduction in resolution not only results in a loss of detail in the image's *texture*, but also helps to "simplify" the geometric models present in the image: even though at full resolution the photograph of the sphere does not appear as a perfect circle but rather as an ellipse, at low resolution the circular model becomes more appropriate.

For this reason, the estimation algorithm is initialized on the edge points in the image at the highest level of the pyramid, identifying the sphere through the (robust) estimation of a circle provided by the *Hough transform* for

[7] These latter can be reduced by applying a *median filter* (which outputs the median of the pixels within a window—e.g., of size 5×5—sliding over the image) before extracting the edges with Canny.

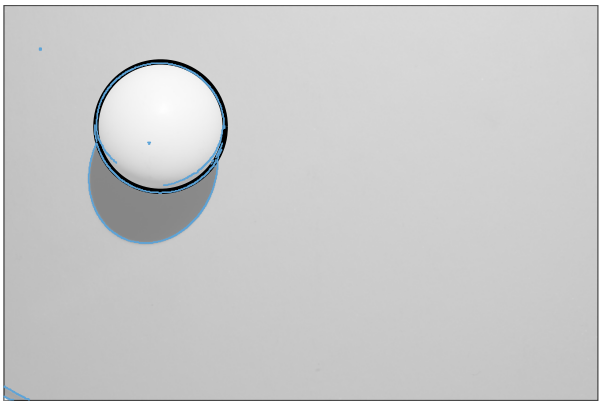


Fig. 6. Circle (highlighted in black) detected using the Hough transform on the image at the highest level of the pyramid.

circles[8], as shown in Fig. 6 by applying the transform to the image in Fig. 5a.

The initialization phase concerns not only the parameters of the ellipse, but also the *scale* $\hat{\sigma}$ of the residuals, which is necessary to assess which points are clear outliers. Although for a circle (with center (x_c, y_c) and radius r) the calculation of the distance of a point from its circumference is straightforward ($d_\perp = \sqrt{(x-x_c)^2 + (y-y_c)^2} - r$), for consistency with the subsequent refinement phase (§ 2.3), the Sampson Error is preferred as an estimate of the distance; recalling Eq. (6), let

$$\delta_n = \frac{\mathbf{q}^\top \mathtt{C}\mathbf{q}}{2\sqrt{(\mathtt{C}\mathbf{q})_1^2 + (\mathtt{C}\mathbf{q})_2^2}}$$

We then apply the procedure in § 2.3.1 to the δ_n values in order to obtain an initial estimate of the scale $\hat{\sigma}$, *excluding from it the points whose distance is greater than half the radius r provided by the detected circle.*

Fig. 7a shows the histogram[9] for the Sampson error computed over the edge points from Fig. 6 with respect to the circle found by the Hough transform: the yellow area marks the range of residuals within $\pm 6\hat{\sigma}$ beyond which a point is considered a clear outlier (see step 3 in the procedure of § 2.3); in particular (Fig. 7b shows in detail the residual associated with each edge point), the outlier points identified in this way roughly correspond to those:

- in the lower-left corner (those with the greatest distances);
- in the upper-left corner (cluster around 0.1);
- the speck of dust inside the circle (the cluster with negative residual);
- part of the sphere's shadow contour.

[8] Algorithm implemented in OpenCV via `cv.HoughCircles`.

[9] The bin widths b_w of the histogram were approximately computed according to the formula $b_w = 2\,\mathrm{IQR}(\mathcal{Q})\,Q^{-1/3}$ [9], where $\mathrm{IQR}(\mathcal{Q})$ is the interquartile range of the sample set $\mathcal{Q} = \{q_i : i = 1 \dots Q\}$, i.e., the 75$^{\text{th}}$ percentile minus the 25$^{\text{th}}$ percentile (the 75$^{\text{th}}$ and 25$^{\text{th}}$ percentiles correspond to the values below which 75% and 25% of the total Q samples lie, respectively, while the median is the 50$^{\text{th}}$ percentile [3, p. 123]).

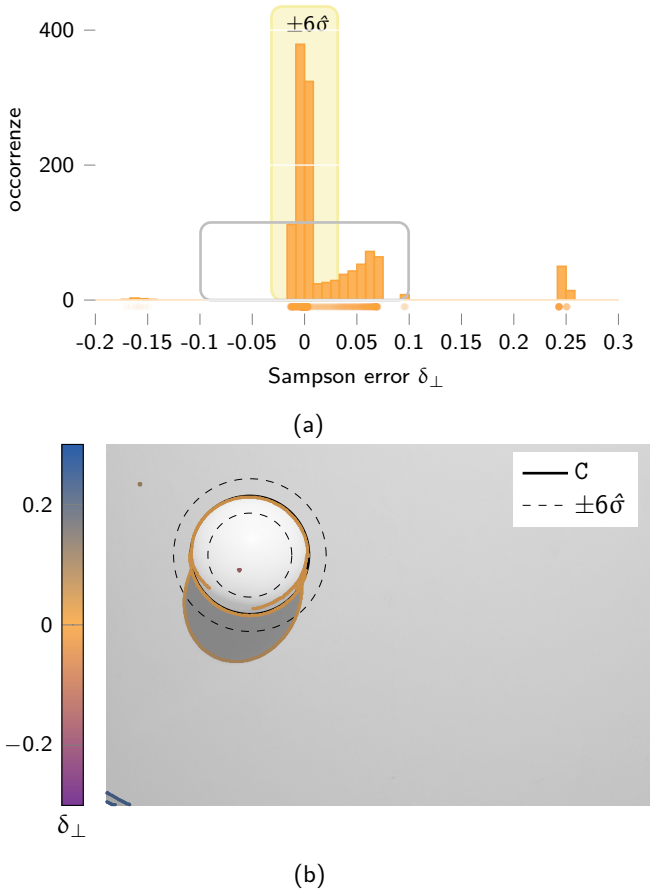


Fig. 7. In (a), the histogram of the Sampson error $\delta_\perp$ over the data from Fig. 6, in normalized coordinates; the scale estimate is $\hat{\sigma} \approx 0.0052671$. In (b), the Sampson error associated with each edge point relative to the detected circle C; the dashed lines indicate the $\pm 6\hat{\sigma}$ band.

The choice of using half the radius as the rejection threshold is rather arbitrary (it could have been, for example, one-third): regardless of the specific value used, this approach eliminates the most evident outliers, reducing the risk that the procedure in § 2.3.1 yields an "incorrect" value (i.e., when outliers account for more than 50% of the points); however, it's also advisable not to set the threshold too low, to avoid the risk of anchoring the estimation to an imprecise initialization (a value of $\bar{\sigma}$ that is too low would, in the subsequent refinement phase, assign very low weights to "correct" points that happen to lie far from the initial circle).[10]

2.3 Refinement

Given a pyramid with L images, proceeding backward from level $L-1$ down to level 0, the refinement procedure for each level l is as follows:

1. Let C be the current estimate of the ellipse provided as input, along with the scale $\hat{\sigma}$ of the residuals

[10] A constant value could also be a viable choice as a rejection threshold (e.g., ±0.1 in normalized pixels), independent of the size of the initial circle.

- if $l = L-1$ (i.e., the highest level of the pyramid), $\mathtt{C}$ is the circle estimate provided by the Hough transform during the initialization phase, as well as the initial estimate of the scale $\hat{\sigma}$;
- for all other levels $0 \leq l < L-1$, $\mathtt{C}$ and $\hat{\sigma}$ are the output of the refinement at level $(l+1)$

2. Extract the edge points in a rectangular neighborhood around the ellipse described by $\mathtt{C}$
3. For these points q_n, evaluate the distance $\delta_n = \delta_\perp(q_n|\mathtt{C})$ and remove from $\{q_n\}$ all points for which $|\delta_n/\hat{\sigma}| > 6$ (clear outliers with respect to the model computed at the previous level), i.e., $\mathcal{Q} = \{q_n : |\delta_n/\hat{\sigma}| \leq 6\}$
4. Set $\mathtt{C}^{(0)} = \mathtt{C}$, $k = 1$
5. Until convergence[11] (i.e., when $\|\mathtt{C}^{(k)} - \mathtt{C}^{(k-1)}\| < \tau$)
   (a) Estimate $\mathtt{C}^{(k)}$ by applying one of the methods in § 2.3.2 to the edge points $q_n \in \mathcal{Q}$, weighted according to a quality index $w_n = w(\delta_n/\hat{\sigma}) \in [0,1]$ (the lower w_n is, the more q_n is considered an outlier; the closer to 1, the more it is considered an inlier)
   (b) Evaluate the distance of the points $q_n \in \mathcal{Q}$ from the ellipse $\mathtt{C}^{(k)}$ as $\delta_n = \delta_\perp\left(q_n|\mathtt{C}^{(k)}\right)$
   (c) Compute $\hat{\sigma}$ by applying the procedure of § 2.3.1 as a robust estimate of the *scale* of the residuals $\{\delta_n\}$

2.3.1 Estimation of $\hat{\sigma}$

The estimation of the scale of the residuals δ_n, $n = 1 \dots N$, is carried out in two stages [10, p. 202], setting $p = 6$ in Eq. (9), 10:[12]

1. an initial estimate is computed according to the formula[13]

$$\hat{\sigma}_\mathrm{i} = 1.4826 \left(1 + \frac{5}{N-p}\right) \sqrt{\mathrm{med}_n \delta_n^2} \tag{9}$$

2. binary weights are assigned

$$m_n = \begin{cases} 1 & \text{for } |\delta_n/\hat{\sigma}_\mathrm{i}| \leq 2.5 \\ 0 & \text{otherwise} \end{cases}$$

3. the scale estimate is finally refined using

$$\hat{\sigma} = \sqrt{\frac{\sum_n m_n \delta_n^2}{\sum_n m_n - p}} \tag{10}$$

[11] Since the matrix of a conic is defined up to a scale factor (possibly negative), the comparison must be made by normalizing $\mathtt{C}^{(k)}$ and $\mathtt{C}^{(k-1)}$: in our case, since we are not dealing with degenerate cases, we can compare the two matrices by dividing their coefficients by the corresponding top-left element (i.e., a in Eq. (2)): $\mathtt{C} \leftarrow \mathtt{C}/a$.

[12] The value of p corresponds to the minimum number of data points required to compute an instance of the mathematical model used: in the case of an ellipse, $p = 6$ points are needed.

[13] The constant 1.4826 is a coefficient that ensures the same *efficiency* as the least-squares method under pure Gaussian noise (the efficiency of a method is defined as the ratio between the lowest attainable variance for the estimated parameters and the actual variance achieved by the method [13]); the factor $1 + 5/(N-p)$ compensates for the effect of having a low number N of data points.

2.3.2 Estimation modes

Two least-squares estimation modes are provided, based on the type of classification (indicated similarly below) for the input points:

- *Binary thresholding*, which applies the algorithm from § 2.4.1 to the points for which $w_n = 1$ (Eq. (11));
- *Gradual thresholding*, which uses the weighted estimation from § 2.4.2 based on the weights w_n given by Eq. (12).

In all cases, the weights are evaluated on the normalized residuals $\bar{\delta}_n = \delta_n/\hat{\sigma}$ with respect to their scale $\hat{\sigma}$.

Binary classification. The binary weights w_n discard all those points whose distance exceeds 3σ (i.e., further outliers beyond those rejected in step 3 of the refinement procedure) from the ellipse estimated in the previous step, considering all remaining points as "pure" inliers:

$$w_\mathrm{B} = \begin{cases} 1 & \text{for } |\bar{\delta}| < 3 \\ 0 & \text{otherwise} \end{cases} \tag{11}$$

Gradual classification. Gradual classification follows *M-estimators*, a family of algorithms that provide an alternative to least squares by using a function $\rho(r)$ instead of r^2 in the definition of the cost function to minimize over the residuals r. M-estimators can be interpreted as a form of weighted estimation (to be iterated as described in step 5 [7, pp. 300–302]), and the function used to compute these weights w_n depends on the specific type of M-estimator. In this work, having selected four different M-estimators [13], the functions used are (Fig. 8):

$$w_\mathrm{H} = \begin{cases} 1 & \text{for } |\bar{\delta}| \leq \kappa = 1.345 \\ |\kappa/\bar{\delta}| & \text{otherwise} \end{cases} \quad \text{(Huber)} \tag{12a}$$

$$w_\mathrm{L} = \frac{1}{\sqrt{1 + \bar{\delta}^2/2}} \quad \text{(L1-L2)} \tag{12b}$$

$$w_\mathrm{T} = \begin{cases} \left(1 - (\bar{\delta}/\kappa)^2\right)^2 & \text{for } |\bar{\delta}| \leq \kappa = 4.685 \\ 0 & \text{otherwise} \end{cases} \quad \text{(Tukey)} \tag{12c}$$

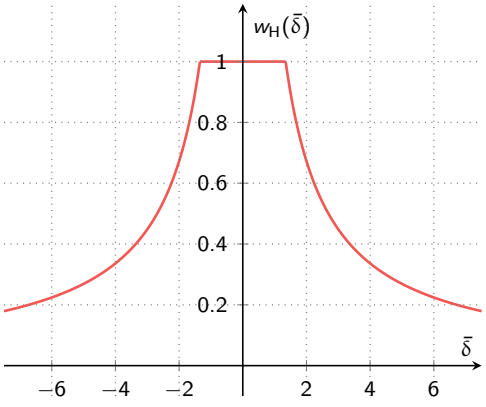
$$w_\mathrm{G} = \frac{1}{(1 + \bar{\delta}^2)^2} \quad \text{(Geman-McClure)} \tag{12d}$$

Observing Fig. 8, it can be seen that for $|\bar{\delta}| > 2$, the weights given by $w_\mathrm{H}(\bar{\delta})$ and $w_\mathrm{L}(\bar{\delta})$ are very similar, whereas those from $w_\mathrm{T}(\bar{\delta})$ and $w_\mathrm{G}(\bar{\delta})$ quickly drop to zero; as a consequence, the behavior of the first two estimators is expected to be comparable, while Tukey and Geman-McClure tends to rely only on points that lie very close to the current ellipse.

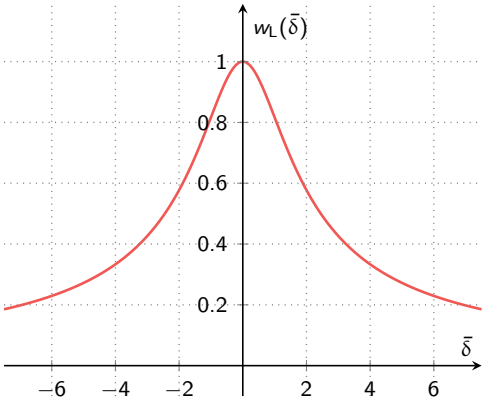
2.4 Direct ellipse fitting

2.4.1 Direct least-squares ellipse fitting

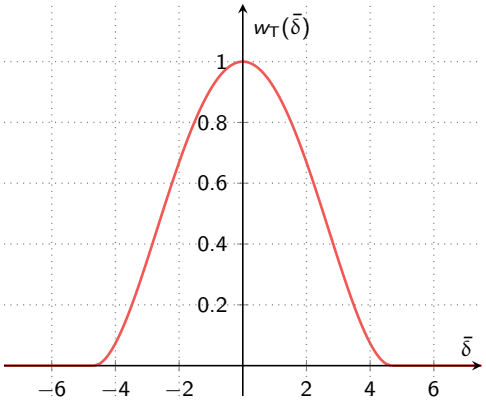
Given a set of N points $\mathcal{Q} = \{q_n = (x_n, y_n) : n = 1 \dots N\}$, the goal is to determine the parameters $a \dots f$ of the ellipse



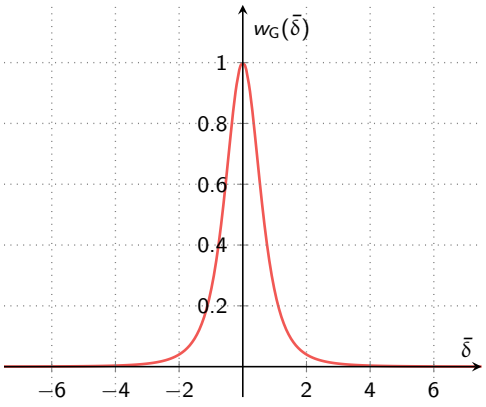
(a) Huber



(b) L1-L2



(c) Tukey



(d) GermanMcClure

Fig. 8. Weight functions used for computing the weights, depending on the chosen M-estimator.

that minimize the algebraic error

$$E(\mathbf{p}|\mathcal{Q}) = \sum_n \underbrace{(ax_n^2 + bx_n y_n + cy_n^2 + dx_n + ey_n + f)}_{\varepsilon_n = \varepsilon(\mathbf{p}|q_n)}{}^2 = \sum_n \varepsilon_n^2$$
$$= \sum_n (\mathbf{d}_n^\top \mathbf{p})^2 = \|\mathtt{D}\mathbf{p}\|^2 = \mathbf{p}^\top \mathtt{D}^\top \mathtt{D}\, \mathbf{p} \tag{13}$$

where the parameters are grouped into the vector $\mathbf{p} = [a\ b\ c\ d\ e\ f]$, with $\mathbf{d}_n = [x_n^2\ x_n y_n\ y_n^2\ x_n\ y_n\ 1]^\top$, and $\mathtt{D} = [\mathbf{d}_1 \dots \mathbf{d}_N]^\top$; the values r_n represent the (algebraic) residuals of the error $E(\mathbf{p}|\mathcal{Q})$.

To avoid the trivial solution $\mathbf{p} = \mathbf{0}$, constraints are imposed on $\mathbf{p}$, exploiting the fact that Eq. (2) is defined up to a scale factor [13]. However, such constraints do not generally ensure that the fitted conic is an actual ellipse.

In many estimation algorithms, the constraint in Eq. (3) is often ignored because it is typically satisfied implicitly—provided that the data used to fit the ellipse are not collinear or nearly so. However, since the conic parameters $\mathbf{p}$ are defined up to a scale factor, it is possible to transform the inequality of Eq. (3) into the equality

$$4ac - b^2 = 1 \tag{14}$$

and include it in the estimation process as a constraint on $\mathbf{p}$ [4][5]:

$$\hat{\mathbf{p}} = \arg\min_{\mathbf{p}} E(\mathbf{p}|\mathcal{Q}) = \arg\min_{\mathbf{p}} \mathbf{p}^\top \underbrace{\mathtt{D}^\top \mathtt{D}}_{\mathtt{S}} \mathbf{p} \quad \text{t. c.} \tag{15a}$$

$$\hat{\mathbf{p}}^\top \underbrace{\begin{bmatrix} 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_{\mathtt{M}} \hat{\mathbf{p}} = 1 \tag{15b}$$

The solution to Eq. (15) can be obtained using Lagrange multipliers, by introducing an additional variable $\lambda \in \mathbb{R}$ into the cost function

$$L(\mathbf{p}, \lambda) = \mathbf{p}^\top \mathtt{S}\mathbf{p} + \lambda(\mathbf{p}^\top \mathtt{M}\mathbf{p} - 1) = \mathbf{p}^\top (\mathtt{S} + \lambda \mathtt{M})\mathbf{p} - \lambda \tag{16}$$

whose partial derivatives, set to 0, lead to

$$\frac{dL}{d\mathbf{p}} = 2(\mathtt{S} - \lambda \mathtt{M})\mathbf{p} = \mathbf{0} \tag{17}$$

$$\frac{dL}{d\lambda} = \mathbf{p}^\top \mathtt{M}\mathbf{p} - 1 = 0 \tag{18}$$

namely

$$\begin{cases} \mathtt{S}\mathbf{p} = \lambda \mathtt{M}\mathbf{p} & \text{(19a)} \\ \mathbf{p}^\top \mathtt{M}\mathbf{p} = 1 & \text{(19b)} \end{cases}$$

Eq. (19a) represents a generalized eigenvalue problem: in general, up to 6 eigenvalue-eigenvector pairs can be obtained[14]; however, only one of them also satisfies the constraint in Eq. (19b). *This corresponds to the only positive*

[14] For $(\mathtt{S} - \lambda \mathtt{M})\mathbf{p} = \mathbf{0}$ to hold for $\mathbf{p} \neq \mathbf{0}$, the matrix $\mathtt{S} - \lambda \mathtt{M}$ must be singular, which is equivalent to requiring its determinant to be zero. Since this matrix has size 6×6, $\det(\mathtt{S} - \lambda \mathtt{M})$ is a sixth-degree polynomial in λ, which can have up to 6 distinct roots.

eigenvalue ($0 < \lambda < \infty$) of Eq. (19a) [4][5]. The solution obtained for Eq. (19a) generally does not satisfy the constraint in Eq. (19b); however, if $(\lambda, \mathbf{p})$ satisfies Eq. (19a), then so does $(\lambda, \eta\mathbf{p})$, which allows us to scale $\eta\mathbf{p}$ such that $\eta^2\mathbf{p}^\top\mathtt{M}\mathbf{p} = 1$ (or[15] $\eta^2\mathbf{p}^\top\mathtt{S}\mathbf{p} = \lambda$). This final step is ultimately irrelevant, since it suffices to satisfy Eq. (3) rather than Eq. (14) (moreover, $\mathbf{p}$ is a homogeneous quantity and represents the same ellipse up to a scale factor).

Since the matrix $\mathtt{M}$ is neither positive definite nor invertible (as defined in Eq. (15)), while $\mathtt{S} = \mathtt{D}^\top\mathtt{D}$ is generally a positive definite matrix, it is preferable to rewrite Eq. (19a) as follows:[16]

$$\mathtt{M}\mathbf{p} = \underbrace{1/\lambda}_{\mu}\,\mathtt{S}\mathbf{p}$$

Also in this case the solution is given by the eigenvector associated with the unique generalized eigenvalue $\mu > 0$.

Data preconditioning. To reduce rounding errors, it is advisable to precondition the data through a transformation[17] $\mathtt{T}$ that maps the coordinates of the point cloud $\mathcal{Q}$ so that they lie within the range $\bar{x}_n, \bar{y}_n \in [-0.5, 0.5]$ [6, Matlab code]:

$$\overbrace{\begin{bmatrix}\bar{x}_n\\ \bar{y}_n\\ 1\end{bmatrix}}^{\bar{\mathbf{q}}} = \overbrace{\begin{bmatrix}\frac{1}{s_x} & 0 & -\frac{m_x}{s_x}\\ 0 & \frac{1}{s_y} & -\frac{m_y}{s_y}\\ 0 & 0 & 1\end{bmatrix}}^{\mathtt{T}} \overbrace{\begin{bmatrix}x_n\\ y_n\\ 1\end{bmatrix}}^{\mathbf{q}} \tag{20a}$$

$$m_x = \frac{\sum_n x_n}{N} \tag{20b}$$

$$m_y = \frac{\sum_n y_n}{N} \tag{20c}$$

$$s_x = \frac{\max(x_n) - \min(x_n)}{2} \tag{20d}$$

$$s_y = \frac{\max(y_n) - \min(y_n)}{2} \tag{20e}$$

To recover the equation of the ellipse in the original data space, we again make use of its matrix representation (Eq. (4)). Let $\bar{\mathtt{C}}$ be the matrix of parameters that solves Eq. (19a) for the points $\bar{\mathcal{Q}} = \{\bar{q}_n = (\bar{x}_n, \bar{y}_n) : n = 1 \dots N\}$. According to Eq. (5), using the inverse transformation $\mathtt{U} = \mathtt{T}^{-1}$ (i.e., $\mathbf{q} = \mathtt{U}\bar{\mathbf{q}}$), we obtain:

$$\mathtt{C} = \mathtt{U}^{-\top}\,\bar{\mathtt{C}}\,\mathtt{U}^{-1} = \mathtt{T}^\top\,\bar{\mathtt{C}}\,\mathtt{T}$$

2.4.2 Direct weighted least-squares ellipse fitting

Assuming that each residual ε_n is associated with a weight w_n reflecting the quality of the point $q_n \in \mathcal{Q} = \{(x_n, y_n) : n = 1 \dots N\}$ (Eq. (12)), we can modify the cost function in Eq. (13) to:

$$E_w(\mathbf{p}|\mathcal{Q}) = \sum_n w_n\varepsilon_n^2 = \sum_n w_n(\mathbf{p}^\top\mathtt{D}_n^\top\mathtt{D}_n\mathbf{p}) = \mathbf{p}^\top\overbrace{\mathtt{D}^\top\mathtt{W}\mathtt{D}}^{\mathtt{S}_\mathtt{W}}\mathbf{p} \tag{21a}$$

$$\mathtt{W} = \begin{bmatrix} w_1 & & & \\ & w_2 & & \\ & & \ddots & \\ & & & w_N \end{bmatrix} \tag{21b}$$

By also retaining the constraint from Eq. (14) ($4ac - b^2 = 1$), the same procedure from § 2.4.1 can be applied to obtain:

$$\begin{cases} \mathtt{S}_\mathtt{W}\mathbf{p} = \lambda\mathtt{M}\mathbf{p} & \text{(22a)}\\ \mathbf{p}^\top\mathtt{M}\mathbf{p} = 1 & \text{(22b)} \end{cases}$$

This system can be solved analogously to Eq. (19).

A Geometric distance from a point to an ellipse

Without loss of generality[18], we consider an ellipse of the form

$$\frac{x}{\alpha^2} + \frac{y}{\beta^2} = 1 \tag{23}$$

For our purposes, a more suitable parameterization of the points (x_e, y_e) on the ellipse given by Eq. (23) is the following[19] with $t \in \mathbb{R}$ [14, pp. 469–470]:

$$x_e(t) = \alpha\frac{1 - t^2}{1 + t^2} \tag{24a}$$

$$y_e(t) = \beta\frac{2t}{1 + t^2} \tag{24b}$$

[15] In [5, eq. 9] there is a typographical error, unlike in [4, eq. 10]. Note that, since $\lambda > 0$ and $\mathtt{S}$ is positive definite (i.e., $\mathbf{p}^\top\mathtt{S}\mathbf{p} > 0\ \forall\mathbf{p} \neq \mathbf{0}$), the value $\eta = \sqrt{\lambda/(\mathbf{p}^\top\mathtt{S}\mathbf{p})} \in \mathbb{R}$.

[16] While in Octave/Matlab the function `[gevecs, gevals] = eig(A,B)` (which solves $\mathtt{A}\mathbf{p} = \lambda\mathtt{B}\mathbf{p}$) allows `B` to be non–positive-definite, the equivalent Python function `gevals, gevecs = scipy.linalg.eigh(A,B)` requires `B` to be positive definite (while `A` only needs to be symmetric). Numerically, Octave's `eig(S,M)` returns 1 negative, 2 positive, and 3 infinite generalized eigenvalues; in contrast, using `eigh(M,S)` in scipy (which solves $\mathtt{M}\mathbf{p} = \frac{1}{\lambda}\mathtt{S}\mathbf{p}$), the eigenvalues $1/\lambda$ for $\lambda = \infty$ are not exactly zero, but take on very small values (on the order of 10^{-16}): the correct positive eigenvalue corresponds to the largest one.

An alternative method is to compute the eigenvalues and eigenvectors of $\mathtt{S}^{-1}\mathtt{M}\mathbf{p} = \mu\mathbf{p}$: the solution corresponds to the only eigenvalue $\mu_i > 0$ of $\mathtt{S}^{-1}\mathtt{M}$ [4, fig. 7]. In Python, both 'scipy.linalg.eig' and 'numpy.linalg.eig' can be used; each function yields 1 positive eigenvalue, 2 negative ones, and 3 zero eigenvalues. The first function returns $\mu \in \mathbb{C}$ in the form $\mu = \nu + 0\imath$, whereas the second returns $\mu \in \mathbb{R}$.

[17] Transformation defined in homogeneous coordinates as $\bar{\mathbf{q}} = \mathtt{T}\mathbf{q}$ according to the matrix $\mathtt{T}$ in Eq. (20).

[18] It is always possible to determine a transformation $\mathtt{T}$ that moves the center of the ellipse to the origin of the reference system and rotates it so that the axes align with those of the Cartesian system; such a transformation is an *isometry*, meaning it preserves distances.

[19] To derive Eq. (24), simply apply the trigonometric substitution formulas in terms of t (i.e., $\cos\vartheta = \frac{1-t^2}{1+t^2}$ and $\sin\vartheta = \frac{2t}{1+t^2}$, where $t = \tan(\vartheta/2)$) to the parametric representation of the ellipse $(x_e(\vartheta), y_e(\vartheta)) = (\alpha\cos\vartheta, \beta\sin\vartheta)$.

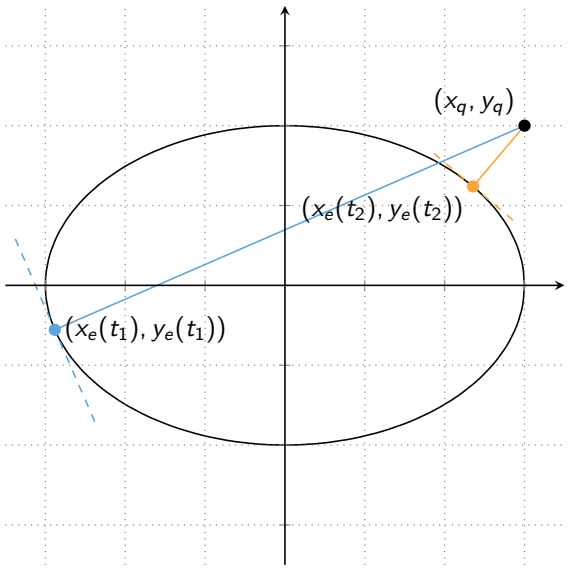


Fig. 9. Computation of the distance from the point $(x_q, y_q) = (3, 2)$ to the ellipse defined by the equation $4x^2 + 9y^2 = -36$: the two highlighted points on the ellipse are the farthest (in blue, for $\hat{t} = t_1$) and the closest (in orange, for $\hat{t} = t_2$); the corresponding distances are $d_1 \approx 6.4131$ and $d_2 \approx 0.9976$, respectively.

Using Eq. (24), the distance from a point $q = (x_q, y_q)$ to a generic point $(x_e(t), y_e(t))$ on the ellipse is

$$\Delta(t|q) = \sqrt{(x_q - x_e(t))^2 + (y_q - y_e(t))^2} \tag{25}$$

To find the point on the ellipse that is closest to q, we search for the minimizer $\hat{t} = \arg\min_t \Delta(t)^2$.

By differentiating $\Delta^2(t|q)$ with respect to t and factoring in t:[20]

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}t}\Delta^2(t|q) &= \frac{\mathrm{d}}{\mathrm{d}t}\left((x_q - x_e(t))^2 + (y_q - y_e(t))^2\right) \\ &= \frac{\mathrm{d}}{\mathrm{d}t}\left[\left(x_q - \alpha\frac{1-t^2}{1+t^2}\right)^2 + \left(y_q - \beta\frac{2t}{1+t^2}\right)^2\right] \\ &= 4\frac{y_q\beta t^4 + 2(x_q\alpha - \beta^2 + \alpha^2)t^3 + 2(x_q\alpha + \beta^2 - \alpha^2)t - y_q\beta}{t^6 + 3t^4 + 3t^2 + 1}\end{aligned}$$

The numerator is a fourth-degree polynomial in t; by setting it to zero we obtain

$$\underbrace{y_q\beta}_{d_4}\, t^4 + \underbrace{2(x_q\alpha + \alpha^2 - \beta^2)}_{d_3}\, t^3 + \underbrace{2(x_q\alpha - \alpha^2 + \beta^2)}_{d_1}\, t + \underbrace{(-y_q\beta)}_{d_0} = 0$$

In general, this equation admits 4 roots, but only 2 of them are real. One corresponds to the point $(x_e(\hat{t}), y_e(\hat{t}))$ on the ellipse closest to q; the other corresponds to the farthest point (see Fig. 9).

[20] Derivations performed using Maxima.

References

[1] P. J. Burt e E. H. Adelson. The laplacian pyramid as a compact image code. *IEEE Transactions on Communications*, 31(4):532–540, 1983.

[2] J. F. Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.

[3] S. Few. *Show Me the Numbers: Designing Tables and Graphs to Enlighten*. Analytics Press, 2012.

[4] A. W. Fitzgibbon, M. Pilu, e R. B. Fisher. Direct least squares fitting of ellipses. In *Pattern Recognition, International Conference on*. IEEE Computer Society, 1996.

[5] A. W. Fitzgibbon, M. Pilu, e R. B. Fisher. Direct least square fitting of ellipses. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 21(5):476–480, 1999.

[6] A. W. Fitzgibbon, M. Pilu, e R. B. Fisher. Direct least square fitting of ellipses. CVonline, 1999.

[7] D. A. Forsyth e J. Ponce. *Computer Vision: A Modern Approach. (Second edition)*. Prentice Hall, 2011.

[8] R. I. Hartley e A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2004.

[9] A. Izenman. Review papers: Recent developments in nonparametric density estimation. *Journal of The American Statistical Association*, 86(413):205–224, 1991.

[10] P. J. Rousseeuw e A. Leroy. *Robust Regression and Outlier Detection*. Wiley Series in Probability and Statistics. Wiley, 1987.

[11] R. Szeliski. Image alignment and stitching: A tutorial. *Foundations and Trends® in Computer Graphics and Vision*, 2(1):1–104, 2007.

[12] S. N. R. Wijewickrema, A. P. Paplinski, e C. E. Esson. Reconstruction of spheres using occluding contours from stereo images. In *18th International Conference on Pattern Recognition (ICPR 2006)*, pp. 151–154. IEEE Computer Society, 2006.

[13] Zhengyou Zhang. *Parameter estimation techniques: A tutorial with application to conic fitting*, volume 15. Elsevier, January 1997.

[14] G. Zwirner e L. Scaglianti. *Strumenti e metodi matematici - Funzioni*. CEDAM - Casa Editrice Dott. Antonio Melani, 1992.